BÖLÜM 10: MODERN UFUKLAR — YAPAY ZEKA ÇAĞINDA MIMARLIK VE KURUMSAL USTALIK

⏱ 9 dk • DERS 39

Parça 39: Eski Sistemleri Modernleştirme Kitabı: Dinozoru Boğmak (Strangler Fig)

20 yıllık kritik bir ana sistem monolitini veri kaybetmeden ve delirmeden taşımak.

🔥 SAHADAN YAŞANMIŞ KRİZ HİKAYESİ

Bir ulusal perakende bankasının ana mevduat sistemi, 1984 yılında yazılmış 4 milyon satırlık COBOL koduna sahip bir IBM ana bilgisayarında (mainframe) çalışıyordu. Orijinal yazarları çoktan emekli olmuştu ve her değişiklik 6 ay test gerektiriyordu. Bir danışmanlık firması 50 milyon dolarlık 'Büyük Patlama (Big Bang)' projesiyle sistemi bir gecede Java'ya geçirmeyi önerdi. Yılbaşı gecesi yeni sistem canlıya alındı; ancak yük altında çöktü ve 800.000 maaş ödemesini reddetti! Banka rezil olarak eski ana bilgisayara geri döndü ve 50 milyon doları çöpe attı. Üç yıl sonra yeni Baş Mimar göreve geldi. Büyük patlama yerine Boğucu İncir (Strangler Fig) kalıbını uyguladı. 24 ay boyunca ekip Change Data Capture ile tek seferde sadece tek bir bankacılık işlevini (önce havale, sonra mobil ödeme, sonra kart işlemleri) dışarı taşıdı. Eski ana bilgisayar iki yılın sonunda tek bir müşteri bile fark etmeden sessizce emekliye ayrıldı.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Büyük Patlama (Big Bang) taşımaları kurumsal mimarlık kariyerlerinin mezarlığıdır. Eski bir sistem ne kadar büyükse, içine o kadar çok örtük iş kuralı, regülasyon istisnası ve unutulmuş hata yaması gömülmüştür. 20 yıllık kurumsal birikimi sıfırdan tek seferde kusursuzca yazmak matematiksel olarak imkansızdır. Tek güvenilir modernizasyon stratejisi **Kademeli Boğma (Incremental Strangling)**: Eski sistemin etrafından trafiği dolandırmak ve yetenekleri katman katman soymaktır.

Temel Mimari İlkeler ve Zihinsel Modeller

4 Adımlı Boğucu İncir Protokolü

1. **Yakala (Intercept):** Eski sistemin önüne bir API Gateway veya tersine vekil sunucu koy.
2. **Dönüştür (Transform):** Tek ve net bir iş yeteneğini (Örn: 'Kullanıcı Girişi') yeni modern mimaride inşa et.
3. **Paralel Çalıştır ve Kıyasla:** İstekleri hem eski hem yeni sisteme kopyala (Shadow Traffic). Otomatik bir kıyaslayıcı ile yanıtların %100 aynı olduğunu canlıda kanıtla.
4. **Geçiş Yap ve Eski Parçayı Sil:** Yönlendiriciyi yeni servise çevir. Bir sonraki modüle geç ve eski dinozorun trafiğini adım adım kes.

Change Data Capture (Debezium ile CDC)

Eski sisteme çift yazma kodu eklemeyin! Veritabanının işlem günlüğünü (WAL) okuyup değişiklikleri anında Kafka'ya akıtan CDC kurarak eski kod tabanına hiç dokunmadan modern veri depolarını güncel tutun.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Büyük Patlama Kesimi:** Gece yarısı şalteri indirip yeni sistemin 15 yıllık eski sistemle tamamen aynı davranacağını umarak dua etmek.

- **Uygulama Katmanında Çift Yazma:** İki sistemin veritabanına aynı anda kod içinden yazmaya çalışıp yarış durumları ve veri bozulması üretmek.
- **Kıyaslama Testi Olmadan Göç:** Gerçek üretim trafiğini paralel çalıştırmadan tahminlere dayalı olarak sistemi canlıya almak.

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Gelen tüm ağ trafiğinin kontrolünü ele geçirmek için eski sistemin önüne bir API Gateway koy.
> - **2. Adım:** 'En İnce Dikey Dilimi' seç: İlk pilot olarak en az riskli ve en bağımsız modülü ayır.
> - **3. Adım:** Canlı trafiği sessizce yeni servise kopyalayan 'Gölge Trafik (Shadow Launch)' kurarak çıktıları otomatik kıyasla.

Mimari Çıkarımlar ve Hile Tablosu

- Asla Büyük Patlama ile sıfırdan geçiş yapmayın; eski sistemi parça parça boğun.
- Gölge trafik ve otomatik kıyaslama sistem denkliğinin matematiksel kanıtıdır.
- Eski sisteme saygı duyun: Çirkin olabilir ama şirketin 15 yıl boyunca maaşlarını o sistem ödedi.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Mevcut monolitinizi yarın boğmaya başlamak zorunda kalsaydınız, ilk ayrıştıracağınız en küçük ve en bağımsız modül hangisi olurdu?